ScribbleDB: Natural Language to Relational Database Synthesis

Aviroop Mitra
aviroopmitra@iisc.ac.in
Indian Institute of Science
Bengaluru, India

Anupam Sanghi
sanghi@iith.ac.in
Indian Institute of Technology Hyderabad
Sangareddy, India

Abstract

[Placeholder ~150 words. Motivate the greenfield database-generation problem: no real data, no formal spec, only a natural-language description. Introduce ScribbleDB as a four-stage agentic pipeline (fact extraction → schema generation → constraint modeling → code generation) whose output is a relational schema (DDL) plus an executable Python program that materializes the populated tables. Highlight the key new contributions versus prior workshop version: (i) deterministic inverse-function code generation for single-dependency DAG nodes, (ii) multivariate distribution support, (iii) ancestral sampling for logical constraints (FA = 1.0), (iv) context enrichment and shard-and-merge for scalability. Close with headline numbers: ~6× table accuracy and ~4× lower latency over SchemaAgent across three benchmark suites.]

ACM Reference Format:
Aviroop Mitra and Anupam Sanghi. 2026. ScribbleDB: Natural Language to Relational Database Synthesis. In *Proceedings of ACM SIGMOD International Conference on Management of Data (SIGMOD '27).* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 Introduction

[Para 1 — Why synthetic databases matter: application development, testing, benchmarking; data-privacy and greenfield constraints prevent access to real data. Emphasise the greenfield setting: no existing deployment, no schema, no seed data.]

[Para 2 — State of the art and its limitations: ab-initio generators (MUDD, PDGF, Myriad) require formal declarative specs; regeneration-based tools (DBSynth, RSGen, Hydra, SAM) require production artefacts. Neither suits a non-technical domain expert with only an NL description.]

*[Para 3 — LLMs as enabler and source of new challenges. Strong NL-to-SQL [**?**] and text-to-schema [**?**] capability. Three core challenges for full DB synthesis: **(a) Expressiveness** (complex cross-table constraints), **(b) Hallucination** (error propagation across pipeline stages), **(c) Scalability** (context window degradation; materialization throughput).]*

[Para 4 — ScribbleDB overview: how it addresses each challenge. Emphasise that the output *is a DDL schema and an executable Python program, not tables directly—separating correctness verification from data materialization.]*

Contributions.

- End-to-end NL-to-database agentic pipeline; outputs relational DDL schema + executable Python code for table materialization.
- *[New]* Deterministic code generation for single-dependency columns in the DAG via inverse functions on join/aggregate tables, reducing LLM calls and guaranteeing correctness-by-construction.
- *[New, partial]* Multivariate distribution support for cross-column correlations beyond univariate priors.
- Shard-and-merge schema generation with Gale–Shapley entity alignment.
- Ancestral sampling for logical constraint satisfaction; FA = 1.0 empirically.
- Comprehensive evaluation: ~6× table accuracy, ~4× lower latency vs. SchemaAgent [**?**] across three benchmark suites.

2 Problem Formulation

Definition 1 (Database Synthesis Problem). *[Formal definition box. Input: an NL description $\mathcal{D}$ of a target database domain plus a target row count n per fact table. Output: (i) a relational schema $\mathcal{S}$ (table definitions, primary keys, foreign keys, data types), and (ii) a Python program $\mathcal{P}$ such that executing $\mathcal{P}$ produces a set of populated CSV/SQL tables satisfying $\mathcal{S}$ and all constraints implied by $\mathcal{D}$. Assumption: $\mathcal{D}$ is internally consistent (no contradiction detection).]*

Correctness Criteria. *[Para: (a) Schema validity: PK uniqueness, FK referential integrity, data-type correctness. (b) Statistical fidelity: generated column distributions match those implied by $\mathcal{D}$ (measured by KS, MRE, NLL). (c) Logical fidelity: all decision-tree and aggregation constraints satisfied (measured by FA). (d) Completeness: recall of schema elements described in $\mathcal{D}$.]*

Running Example. *[Para: Introduce the* Credit Risk (Loan Origination) *database as the running example throughout the paper. Summarise the NL description: applicants with credit scores (300–850), annual income, DTI ratio; approved loans with an interest rate derived from a tiered lookup of credit score and loan amount (e.g., credit score $>$ 750 & amount $<$ \$50K $\Rightarrow$ rate = 3.5%); overall interest rate follows a Gaussian distribution. This example is used in all stage-level figures and the case study in §8.]*

3 System Architecture

[Para 1: Four-stage pipeline overview: Fact Extraction → Schema Generation → Constraint Modeling → Code Generation. Each stage uses deterministic guard rails; failures trigger a bounded repair loop (max 5 retries per stage). The pipeline outputs DDL + a Python program, not tables directly, separating synthesis from materialization.]

[Para 2: Inter-stage interfaces. $\mathcal{F}$ = enriched fact set (JSON list of atomic statements with tags and external-kind annotations). $\mathcal{S}$ = relational schema (JSON: tables, columns, PKs, FKs, types). $\mathcal{C}$ = structured constraints (nested JSON: distributions, decision trees, aggregations). $\mathcal{G}$ = column-level dependency DAG (adjacency list). $\mathcal{P}$ = Python program (plain text).]

SIGMOD '27, TBD
2026. ACM ISBN XXX-X-XXXX-XXXX-X/27/00
https://doi.org/10.1145/XXXXXXX.XXXXXXX

*[Figure 1 placeholder (full-width): Three panels. **Left**: NL description of the Credit Risk (Loan Origination) database. **Middle**: generated DDL schema excerpt (Applicants, Loans tables with PKs, FKs). **Right**: excerpt of rows from the materialized Loans table with the tiered interest-rate constraint annotated. Illustrates the end-to-end transformation.]*

Figure 1: ScribbleDB converts an NL description (left) into a relational DDL schema (middle) and an executable Python program whose output satisfies the stated constraints (right). Running example: Credit Risk (Loan Origination) database.

[Figure 2 placeholder (full-width): ScribbleDB architecture. Four stage boxes arranged left-to-right with labelled inter-stage arrows ($\mathcal{F}$, $\mathcal{S}$, $\mathcal{C}$+$\mathcal{G}$, $\mathcal{P}$). Inside each box: LLM agent icon(s) + deterministic validator block + repair-loop arrow. Final outputs annotated: DDL schema (SQL file) and Python materialization program.]

Figure 2: ScribbleDB architecture. Four stages connected by typed inter-stage interfaces. Each stage is guarded by a deterministic validator and a bounded repair loop. Final output: relational DDL schema and executable Python program.

4 Fact Extraction

The initial input to the synthesis pipeline is a paragraph of natural-language prose ($\mathcal{D}$). Fact Extraction transforms $\mathcal{D}$ into a set of discrete, *atomic facts* ($\mathcal{F}$). By decomposing the text, removing ambiguities, and resolving information deficiencies through targeted enrichment, this stage ensures the domain is rigorously defined before any structural modelling begins. Furthermore, extracting requirements as atomic units provides strict provenance, enabling every subsequent schema decision to be traced directly back to the specific evidence that justified it.

4.1 Atomic Extraction and Verification

An atomic fact encapsulates exactly one requirement: the existence of an entity, a specific attribute, a relationship, or a logical rule. For example, the statement "An applicant has a credit score between 300 and 850" is decomposed into two distinct facts: (1) an applicant has a credit score, and (2) a credit score lies between 300 and 850. This strict atomicity decouples structural definitions from logical constraints.

To ensure traceability, every fact is anchored to the specific span of $\mathcal{D}$ that justifies it. Instead of relying on an LLM to unreliably output character offsets, the extractor returns an ordered list of verbatim substrings from $\mathcal{D}$, with the atomic facts nested beneath them. Precise character offsets are then recovered via a deterministic scan.

Because initial extraction requires semantic judgement, it is delegated to an LLM agent. However, to guarantee fidelity, the output is iteratively scrutinized by a localized *Dual Guard* repair loop (Figure 3, top). The first guard is a deterministic Validator that enforces structural integrity by confirming that each fact's origin segment is genuinely a verbatim substring of $\mathcal{D}$. The second guard is an LLM-based Verifier that identifies semantic omissions or hallucinated details. The extractor edits only the faulted facts until the audit returns clean or a retry budget is exhausted.

4.2 Gap-Directed Context Enrichment

Even a fact set that is perfectly faithful to $\mathcal{D}$ may be too sparse to induce a usable schema, as initial descriptions routinely omit

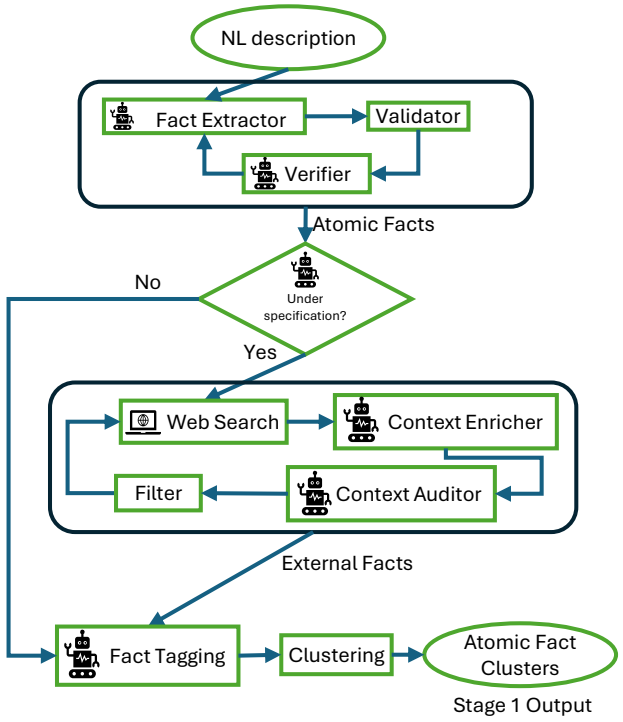

Figure 3: The Fact Extraction stage. A verified set of atomic facts is gated on completeness; under-specified inputs are enriched against retrieved web evidence under a semantic auditor and a deterministic structural filter before all facts are tagged and clustered. The stage emits clusters of atomic facts, the sole input to Schema Generation (§5).

foundational domain knowledge. A completeness agent evaluates the verified facts against the expected structural skeleton for the target domain. If it detects severe omissions (e.g., missing core entities), it emits typed *coverage gaps* and routes the facts into an enrichment loop (Figure 3, center).

Enrichment augments the dataset with external facts, introducing a risk of hallucination. To mitigate this, a Web Search module retrieves external evidence and populates a shared pool with stable citation tags. A ContextEnricher proposes new facts by citing these tags. A semantic ContextAuditor dynamically checks every proposed fact against the retrieved text, immediately rejecting any that are ungrounded. Finally, a deterministic filter ensures the new facts anchor cleanly to the original description. The loop terminates once all severe gaps are closed and the fact set is structurally sound.

4.3 Tagging and Fact Clustering

The verified and enriched facts are first classified by a Tagger into four categories: structural (entities/attributes), logical (rules), statistical (distributions), and metadata. These tags route facts to the appropriate downstream stages.

To enable parallel schema generation, the comprehensive fact set must then be partitioned into independent groups. Rather than invoking an LLM, the partitioning is achieved mechanically by treating the provenance segments recovered during initial extraction as indivisible atomic units (Algorithm 1). By grouping facts mechanically based on their origin, the system preserves the local context intended by the original author.

The algorithm frames clustering as a Dirichlet Process Mixture Model driven by three complementary signals: *positional adjacency* (segments close together in the text), *semantic similarity* (embedding proximity), and *cross-references* (explicit structural hints). Using a collapsed Gibbs sampler, the algorithm auto-calibrates the relative weights of these signals and extracts stable clusters from the consensus of posterior samples, ensuring robust partitions that serve as the definitive output of the stage.

5 Schema Generation

Once the textual domain description is distilled into robust, isolated clusters of atomic facts, the pipeline must translate them into a unified relational schema. Bridging the gap between natural language constraints and strict relational algebra in a single step forces an LLM to simultaneously reason about domain semantics and mechanical database normalization. This routinely causes hallucinated keys, dropped requirements, and structurally malformed outputs.

To systematically decouple these concerns, Schema Generation employs a parallel *shard-and-merge* architecture (Figure 4). The process first elevates each fact cluster into a localized conceptual model, bypassing rigid relational rules. These conceptual shards are then merged globally using a deterministic mathematical engine that flags contradictions for targeted agentic adjudication. Only when the global conceptual model is perfectly consistent is it mapped deterministically into a SQL-compliant relational schema, which is subjected to a final utility audit.

5.1 Parallel Conceptual Modelling

The input to this phase is the set of independent fact clusters produced in Stage 1. Because these clusters are structurally isolated by design, they can be processed in parallel. For each cluster, an ER Extractor agent is tasked with generating a local Entity-Relationship (ER) shard comprising entities, attributes, and relationships. By restricting the LLM to a purely conceptual vocabulary—omitting foreign keys, indexing, and data types—the model is forced to focus strictly on domain semantics rather than implementation details.

To guarantee that the resulting ER shard faithfully reflects its source facts, the extraction runs within a localized *Dual Guard* repair loop (the top box of Figure 4):

(1) **Conceptual Filter (Deterministic):** The draft shard is immediately evaluated by a mechanical filter that enforces strict invariants without invoking the LLM. It verifies that every entity and attribute maps directly to a provided fact ID and that no syntactically malformed relationships exist. If the filter detects an anomaly, it short-circuits the loop, returning a deterministic error trace directly to the extractor.
(2) **ER Auditor (Semantic):** Once the draft passes the mechanical filter, it is evaluated by an LLM-based auditor. The auditor cross-references the shard against the atomic facts of that cluster to catch semantic omissions—such as dropped logical

Algorithm 1 Segment-Grounded Fact Clustering

Require: Atomic facts $\mathcal{F}$ anchored by provenance, Sampling iterations N
Ensure: Disjoint fact clusters (shards) C

Phase 1: Feature Extraction
Group $\mathcal{F}$ into indivisible text segments based on their character positions in the source document
$\mathbf{S}_{sim} \leftarrow$ Compute semantic similarity matrix using text embeddings of the segments
$\mathbf{S}_{adj} \leftarrow$ Compute positional proximity matrix $(1/(1 + \text{text\_distance}))$
$\mathbf{R} \leftarrow$ Extract binary matrix of explicit cross-references between segments
Phase 2: Iterative Cluster Refinement (Gibbs Sampling)
Initialize each segment into its own isolated cluster
for iteration $t = 1$ **to** N **do**
    $\mathbf{W} \leftarrow$ Dynamically fuse $\mathbf{S}_{adj}$ and $\mathbf{S}_{sim}$ using auto-tuned mixing weight λ
    for each segment i in a randomized order **do**
        Temporarily remove i from its current cluster
        for each active cluster k, calculate affinity score based on:
            - The structural density of $\mathbf{W}$ between i and all members of k
            - A strong reward for any direct cross-references $\mathbf{R}$ with members of k
        Calculate the baseline penalty for isolating i into a newly formed cluster
        Probabilistically reassign i to a cluster based on normalized affinity scores
    end for
    if t is a calibration interval **then**
        Re-calibrate λ and internal affinity parameters based on current cluster densities
    end if
    Record segment co-occurrences for this iteration (post-burn-in)
end for
Phase 3: Consensus Extraction
$\mathbf{M} \leftarrow$ Build co-association matrix (frequency of segments sharing a cluster across all iterations)
Partition $\mathbf{M}$ at a 50% consensus threshold to finalize stable segment clusters
$C \leftarrow$ Map the grouped segments back to their constituent atomic facts $\subset \mathcal{F}$
return C

constraints or hallucinated relationships that the mechanical filter cannot detect.

If either guard fails, the localized critique is appended to the extractor's prompt for a retry. A conceptual shard only exits this loop when it satisfies both guards, ensuring structural validity and perfect provenance tracking before global merging.

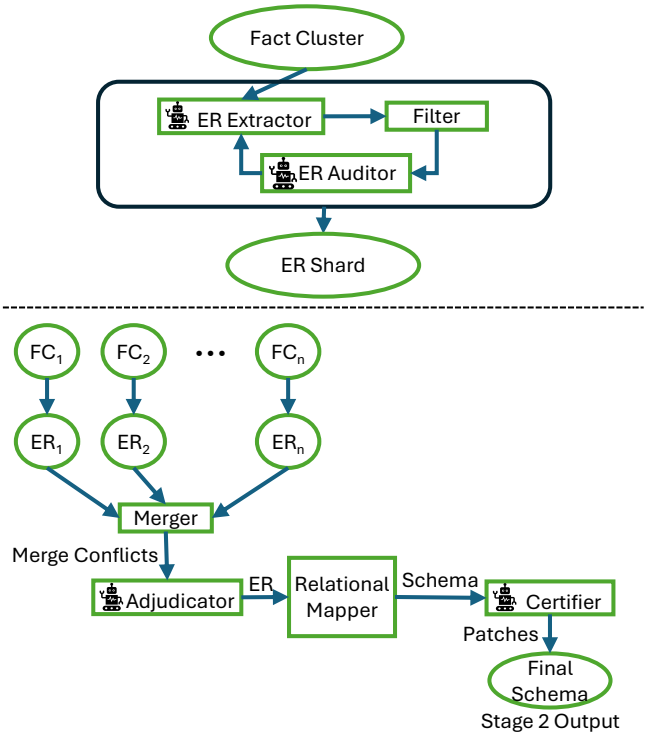

Figure 4: The Schema Generation stage. Independent fact clusters are transformed into conceptual shards via a dual-guard repair loop. Shards are deterministically merged, with conflicts resolved by an Adjudicator. The unified conceptual model is mechanically mapped to a relational schema and audited by a Certifier.

5.2 Multi-Verse Merging and Conflict Adjudication

While the individual conceptual shards are internally consistent, they often contain redundant or conflicting representations of the same real-world concepts (e.g., one shard defining a `Customer` entity while another defines a `Client`).

Rather than relying on an LLM to blindly guess how to merge the entire fragmented domain, the system applies a deterministic mathematical engine to perform *Multi-Verse Correlation Clustering* (Algorithm 2). This MERGER systematically compares entities and relationships across all shards, fusing exact matches and structurally identical nodes. Crucially, when the engine detects semantic or structural disagreements (such as differing cardinality rules or naming overlaps), it does not attempt to resolve them. Instead, it pauses the merge and emits explicit *Conflict Flags*.

These tensions form a *Conflict Graph*, where nodes represent conflicting entities and edges denote the contradictions between them. The system extracts the connected components of this graph and dispatches an ADJUDICATOR agent to resolve each isolated subgraph in parallel.

The Adjudicator reviews the conflicting structures alongside their original grounding facts and issues structured resolution

Algorithm 2 Multi-Verse Correlation Clustering

Require: Local conceptual shards $\mathcal{M}_1, \ldots, \mathcal{M}_k$
Ensure: Unified conceptual model $\mathcal{M}_{\text{global}}$, Conflict flags F
Phase 1: Semantic Scoring
Flatten all entities across all shards into a global pool $\mathcal{E}$
Compute pairwise semantic similarity scores between all cross-shard entities (using embeddings of attributes and grounding facts)
Convert similarities into a probability matrix $\mathbf{P}$ representing the likelihood that two entities are the same concept
Heavily boost $\mathbf{P}_{ij}$ if two cross-shard entities share an exact name
Phase 2: Global Clustering
Construct a weighted graph where edge weight $\mathbf{W}_{ij} = \mathbf{P}_{ij} - 0.5$ (positive for likely merges, negative otherwise, and $-\infty$ for intra-shard pairs)
Partition the graph into clusters C by maximizing the sum of within-cluster edge weights (Correlation Clustering)
Phase 3: Tension Flagging
Initialize conflict flags $F \leftarrow \emptyset$
for each cross-shard entity pair (i, j) **do**
    if they strongly wanted to merge ($\mathbf{P}_{ij} > 0.5$) but ended up in different clusters **then**
        Add `VETOED_MERGE` flag to F
    else if they wanted to remain separate ($\mathbf{P}_{ij} < 0.5$) but were dragged into the same cluster **then**
        Add `FORCED_MERGE` flag to F
    end if
end for
Phase 4: Unification and Validation
Form unified entities by fusing all attributes and source facts within each cluster
Flag `IDENTIFIER_DISAGREEMENT` if fused entities specify conflicting primary keys
Overlay all relationships and deduplicate identical edges
Flag `CARDINALITY_CONTRADICTION` if duplicate relationships specify conflicting constraints (e.g., 1:N vs. M:N)
Extract `ATTR_SYNONYM` and `CROSS_CATEGORY` flags via a secondary probabilistic pass over attribute and relationship names
return Unified $\mathcal{M}_{\text{global}}$ and flags F

Algorithm 3 Deterministic Relational Mapping

Require: Unified conceptual model $\mathcal{M}$ (Entities E, Relationships R, FDs)
Ensure: Relational schema $\mathcal{S}$ (Tables, Columns, Constraints)
Initialize $\mathcal{S} \leftarrow \emptyset$
for each entity $e \in E$ **do**
    Base columns $\leftarrow$ all scalar attributes of e
    PK $\leftarrow$ derive from functional dependencies or defined identifiers
    if PK is invalid or missing **then**
        Synthesize surrogate integer PK (e.g., `e_id`)
    end if
    Add table $\mathcal{T}_e$ to $\mathcal{S}$
    for each multivalued attribute a in e **do**
        Add child table $\mathcal{T}_{e,a}$ to $\mathcal{S}$ with FK referencing $\mathcal{T}_e$.PK
    end for
end for
for each relationship $r \in R$ **do**
    if r is M:N or n-ary **then**
        Name junction table via derived noun rules (avoiding verb collisions)
        Add junction table with FKs referencing all participant tables
    else if r is 1:N **then**
        Add FK column(s) to the child table referencing the parent table
    else if r is 1:1 **then**
        Add FK to one participant and enforce `UNIQUE` constraint
    end if
end for
for each weak entity $e \in E$ **do**
    Append owner table's PK to $\mathcal{T}_e$ as a composite PK and FK
end for
$\mathcal{S} \leftarrow$ NormalizeAndAlign($\mathcal{S}$) ▷ Align data types, wire orphans
Run deterministic repair loop to prune structurally hollow tables
return $\mathcal{S}$

patches (for example, merging entities, renaming attributes, or resolving cardinality). These patches are deterministically applied to fix the global conceptual model, ensuring that complex conflict resolution is handled selectively and explicitly by the LLM, rather than burying it in a massive generation step.

5.3 Deterministic Mapping and Certification

Once the global conceptual model is deduplicated and conflict-free, it must be lowered into an executable relational schema. Because the conceptual model is already structurally perfect, this translation does not require an LLM.

Instead, a deterministic Relational Mapper mechanically applies standard database design rules (Algorithm 3). Entities are mapped to tables, multi-valued attributes to junction tables, and conceptual relationships to primary and foreign key constraints. This mechanical lowering completely eliminates the hallucinated keys and circular dependencies that plague single-step LLM schema generators.

Finally, the relational schema undergoes a safety audit by a Schema Certifier agent. The Certifier evaluates the schema against the overarching analytical goal and the full set of facts to ensure that all required metrics and reporting queries can be satisfied. If the schema is structurally sound but analytically deficient, the Certifier emits rigid structural patches to append the missing tables or columns, resulting in the definitive global schema.

Algorithm 4 Deterministic Decision-Tree Feasibility Check

Require: Set of decision-tree constraints $T \subseteq C$
Ensure: Feasibility verdict per constraint; infeasibility report if any
1: *[Placeholder: per constraint—enumerate leaf conditions, check contradictory conjunctions (e.g., $x > 750$ AND $x < 300$), verify outcome type compatibility, check referenced columns exist in $\mathcal{S}$; flag infeasible constraints for LLM repair]*

6 Constraint Modeling

6.1 Structured Constraint Representation

[Para: All constraints are stored in a nested structured format (JSON): (a) univariate distributions (type, parameters, target column); (b) decision-tree logical constraints (if–then rules over column values, including cross-table aggregations such as SUM, MAX); (c) multivariate distribution groups (§6.3). This representation enables downstream deterministic validation and code generation.]

6.2 Constraint Extraction and Feasibility Verification

[Para: The global schema graph is sharded into connected components. A constraint-modeling agent extracts structured constraints per component in parallel. A mathematics agent then verifies distribution feasibility (e.g., checking that parameter values are in the valid domain for the specified distribution family). Decision-tree feasibility is checked deterministically (Algorithm 4).]

6.3 Multivariate Distribution Support

[Para — PLACEHOLDER (not fully fleshed out): Extension beyond univariate constraints to capture cross-column correlations. Planned approach: [TBD — likely copula-based joint sampling (Gaussian or vine copulas) or a conditional chain representation where later columns in the DAG topological order are sampled conditionally on earlier ones]. Describe: (i) the structured representation for multivariate groups in C; (ii) how multivariate group nodes are inserted into the dependency graph $\mathcal{G}$; (iii) how the code generator handles a multivariate group node. This section will expand substantially once the approach is finalised.]

6.4 Dependency Graph Construction

[Para: From FK links and structured constraints, a column-level DAG $\mathcal{G}$ is built in reverse-causal *order: constrained outcome columns (e.g.,* `interest_rate`*) are positioned as roots; their independent drivers (e.g.,* `credit_score`, `loan_amount`*) are leaves. Multivariate group nodes are inserted as composite nodes (TBD). Cycle detection is performed via topological sort; if a cycle is detected, a patch agent updates C to break the cycle and the graph is rebuilt.]*

7 Code Generation

7.1 Deterministic Inverse-Function Code for Single-Dependency Columns

[Para: Key new contribution. For each column node $c \in \mathcal{G}$ with exactly one *incoming dependency edge, Python code is synthesised*

Algorithm 5 Deterministic Dependency DAG Construction

Require: Schema $\mathcal{S}$, structured constraints C
Ensure: Column-level DAG $\mathcal{G}$
1: *[Placeholder: one node per column; FK edges reverse-causal; constraint edges outcome ← driver; multivariate group nodes (TBD); topological sort; cycle → patch agent updates C, rebuild]*

[Figure 5 placeholder: Column-level DAG for Credit Risk. Solid nodes: `credit_score` *(leaf),* `loan_amount` *(leaf),* `interest_rate` *(root). Solid directed edges:* `credit_score` → `interest_rate`, `loan_amount` → `interest_rate`*. FK edges shown as dashed. Dashed composite node group for a multivariate dependency (placeholder). Node colouring: deterministic-path nodes (blue), LLM-path nodes (orange).]*

Figure 5: Dependency DAG for the Credit Risk running example. `interest_rate` **is the reverse-causal root; its drivers are leaves. Node colour indicates the code-generation path taken in Stage 4.**

Algorithm 6 Stage 3: Constraint Modeling

Require: Schema $\mathcal{S}$, fact set $\mathcal{F}$
Ensure: Structured constraints C, dependency DAG $\mathcal{G}$
1: *[Placeholder: shard $\mathcal{S}$ into connected components → parallel constraint-modeling agents → math agent distribution feasibility → DTFeasibilityCheck (Alg. 4) → BuildDAG (Alg. 5) → return $C, \mathcal{G}$]*

Algorithm 7 Deterministic Inverse-Function Code Synthesis

Require: Single-dependency column c, aggregation type τ, constraint ϕ
Ensure: Python code snippet p_c
1: *[Placeholder: synthesise intermediate aggregate/join table; dispatch on τ: SUM → uniform partition, MAX → argmax selection, AVG → constrained sampling, JOIN → FK-key draw; unsupported τ falls through to LLM path (§7.2)]*

deterministically—*no LLM call is made. The strategy is: (i) synthesise an intermediate join or aggregate table satisfying the aggregate constraint (the "forward" computation); (ii) decompose back to base column values using an inverse function whose form is fully determined by the aggregation type. This guarantees correctness-by-construction for the covered columns and eliminates LLM API cost for these nodes. Covered aggregation types and their inverses: SUM → uniform partition, MAX → argmax selection, AVG → constrained sampling around mean, JOIN (FK assignment) → deterministic FK-key draw.]*

Algorithm 8 LLM Ancestral Sampling for Intertwined Constraints

Require: Intertwined column c, its local sub-DAG, constraints $C_c \subseteq C$
Ensure: Python code snippet p_c
1: *[Placeholder: build prompt (sub-DAG + C_c + generation order + template) → LLM synthesises inverse function → deterministic sanity check (type compatibility, no undefined variables) → repair loop max 5 retries]*

[Figure 6 placeholder: Decision flowchart. Input: column node c from $\mathcal{G}$. Diamond: "single incoming dependency?" Yes → Alg. 7 (deterministic inverse, labelled with aggregation type). No → Alg. 8 (LLM ancestral sampling). Illustrated on Credit Risk DAG: `interest_rate` *routed to LLM (two drivers);* `loan_amount` *FK reference routed to deterministic JOIN inverse.]*

Figure 6: Code generation routing per column node in $\mathcal{G}$. Single-dependency columns take the deterministic inverse path (no LLM call); intertwined columns are handled by the LLM ancestral-sampling agent.

7.2 LLM Ancestral Sampling for Intertwined Constraints

[Para: When a column node has multiple incoming dependency edges—i.e., its value is governed by constraints involving several other columns simultaneously—no clean closed-form inverse exists. In this case, a code-generator LLM agent is invoked. The agent receives the local sub-DAG centred on c, the relevant constraints from C, and the pre-determined generation order from $\mathcal{G}$. It synthesises an inverse function *generatively: given already-sampled values for the driver columns, it emits Python code that samples c such that the constraints are satisfied. By following the reverse-causal order of $\mathcal{G}$, this scheme guarantees constraints are satisfied* by construction*, with no post-hoc repair.]*

7.3 Code Merge and Smoke Testing

[Para: Shard-level Python snippets are merged into a single program using a fixed code template with five sections: imports, seed definitions, helper functions, intermediate-table synthesis, base-table materialisation. When two snippets contain conflicting initialisations of the same column (e.g., one uses a distribution spec, another uses random initialisation), a deterministic tie-breaking rule resolves the conflict (distribution-spec $\succ$ random; structured $\succ$ unstructured). The merged program is smoke-tested by running it at 10% of the target row count. Any runtime failure triggers an agent repair loop before full-scale materialisation. The final output is $\mathcal{P}$.]

Algorithm 9 Deterministic Code Merge

Require: Shard-level code snippets $\{p_1, \ldots, p_k\}$, code template $\mathcal{T}$
Ensure: Consolidated Python program $\mathcal{P}$
1: *[Placeholder: align snippets to template sections (imports / seeds / helpers / intermediate / base); resolve column initialisation conflicts via tie-breaking rules; smoke test at 10% scale; repair loop max 5 retries on failure]*

Algorithm 10 Stage 4: Code Generation

Require: Schema $\mathcal{S}$, constraints C, DAG $\mathcal{G}$
Ensure: Python program $\mathcal{P}$
1: *[Placeholder: traverse $\mathcal{G}$ in reverse-topological order; single in-edge → DetInverseCode (Alg. 7), multiple in-edges → LLMAncestralSampling (Alg. 8); CodeMerge (Alg. 9) → return $\mathcal{P}$]*

8 Experimental Evaluation

8.1 Experimental Setup

Datasets. *[Placeholder: (a) RSchema [**?**]: 381 NL-to-schema test cases (schema only, no data). (b) Handcrafted [**?**]: 135 test cases (NL, schema, distributional and logical constraints) generated via Claude 4.5 Sonnet, spanning Credit Risk, Quantitative Finance, Clinical Trials; 3–50 tables; 10K–1M rows per fact table. 2:3:5 ratio across small (<10), medium (11–25), large (26+) schemas. (c) Benchmarks: IMDB [**?**], TPC-H [**?**], TPC-DS [**?**], MIMIC-IV [**?**] official documentation converted to NL descriptions that deliberately omit benchmark names, specific schema details, and explicit parameters.]*
LLM & Baseline. *[Placeholder: GPT-4o for all LLM components of ScribbleDB. Baseline for schema quality: SchemaAgent [**?**]. No prior baseline for data modeling quality (ScribbleDB is first to target full synthesis).]*
Metrics. *[Placeholder: Schema: Table/Attr/PK/DT F1 and Accuracy. Statistical fidelity: MRE↓, NLL↑, KS↓. Logical fidelity: FA↑. Efficiency: schema gen tokens/time, per-row gen time, avg. retry loops. New metric:* Deterministic Path Ratio *(§8.7) — fraction of DAG nodes routed to deterministic vs. LLM code generation.]*

8.2 Schema Quality

[Placeholder: ScribbleDB achieves near-perfect Table F1 (95–100) across all three dataset groups. SchemaAgent degrades sharply on complex schemas (Table F1 drops to 48–52 on Handcrafted/Benchmark). Key insight: sharding enables parallel agents to focus on smaller schema portions; combined with context enrichment, it recovers schema elements that SchemaAgent misses when the NL is ambiguous or incomplete.]

8.3 Data Modeling Quality

[Placeholder: Statistical realism (KS < 0.3, MRE < 0.2, NLL > 0.7) and logical fidelity (FA = 1.0) across Handcrafted and Benchmark datasets. Metrics are schema-recall-penalised to account for structural mismatches. Note: multivariate evaluation metrics will be added in this table once §6.3 is finalised—a new column (e.g., copula KS or rank correlation error) is reserved.]

Table 1: Schema quality comparison. Best result per metric/dataset in bold.

Dataset	System	TF1	TAcc	AF1	AAcc	PKAcc	DTAcc
RSchema	SchemaAgent				*[values]*		
	ScribbleDB				*[values]*		
Handcrafted	SchemaAgent				*[values]*		
	ScribbleDB				*[values]*		
Benchmark	SchemaAgent				*[values]*		
	ScribbleDB				*[values]*		

Table 2: Data modeling quality. Metrics penalised by schema recall fraction. *[TBD: multivariate column to be added once §7.3 is finalised.]*

Dataset	MRE↓	NLL↑	KS↓	FA↑
Handcrafted		*[values]*		
TPC-H		*[values]*		
TPC-DS		*[values]*		
IMDB		*[values]*		
MIMIC-IV		*[values]*		

[Figure 7 placeholder: Three panels (one column each). Each: generated histogram (blue) overlaid with ground-truth density curve (orange). Panel 1: normally distributed column (e.g., income). Panel 2: Zipfian column (e.g., transaction count). Panel 3: bounded column (e.g., credit score 300–850). All from Handcrafted dataset.]

Figure 7: Distribution fidelity: generated vs. ground-truth density for three representative columns in the Handcrafted dataset.

8.4 Scalability Analysis

[Placeholder: Schema-generation latency scales sub-linearly with table count for ScribbleDB (parallel sharding) vs. linearly for SchemaAgent (sequential). 3–4× speedup on complex schemas. Row-generation time scales linearly with target row count at 10–30 ms/row, independent of schema complexity (deterministic and LLM code paths both amortise to per-row costs).]

8.5 Ablation Studies

[Placeholder: Quantify contribution of each architectural component on the Handcrafted dataset. New ablation row: "w/o Det. CodeGen" replaces all deterministic code-generation paths with LLM calls—expected to show higher token cost, higher latency, and potentially lower FA if the LLM occasionally fails to satisfy constraints exactly.]

Table 3: Ablation on Handcrafted dataset. Best result per column in bold.

Configuration	TF1	TAcc	AF1	MRE	NLL	KS	FA
Full ScribbleDB				*[values]*			
w/o Enrichment				*[values]*			
w/o Sharding				*[values]*			
w/o Anc. Sampling				*[values]*			
w/o Det. CodeGen				*[values — new row]*			

Table 4: Cost and efficiency (avg. per test case).

Dataset	System	Tokens	Sch. Time	Row Time	Retries
RSchema	SchemaAgent		*[values]*		
	ScribbleDB		*[values]*		
Handcrafted	SchemaAgent		*[values]*		
	ScribbleDB		*[values]*		
Benchmark	SchemaAgent		*[values]*		
	ScribbleDB		*[values]*		

8.6 Cost and Efficiency

8.7 Deterministic Code Generation Analysis

[Placeholder: Characterise the new deterministic code-generation contribution. Metrics to report: (a) Deterministic Path Ratio (DPR)*: fraction of DAG column nodes routed to Alg. 7 vs. LLM (Alg. 8). Expect DPR ≈ 40–60% on Handcrafted dataset (hypothesis: many FK and simple aggregate columns are single-dependency). (b) LLM call reduction: absolute and relative number of LLM calls saved vs. an all-LLM baseline. (c) Correctness delta: compare FA for deterministic-path columns vs. LLM-path columns— deterministic path expected to achieve FA = 1.0 by construction.]*

8.8 Case Study: Credit Risk Database

[Placeholder: End-to-end walkthrough. NL description (§4) → 9 atomic facts tagged STRUCTURAL/LOGICAL/STATISTICAL (Stage 1) → DDL schema: Applicants (ApplicantID PK, CreditScore, Income, DTI), Loans (LoanID PK, ApplicantID FK, LoanAmount, InterestRate) (Stage 2) → dependency DAG: `interest_rate` ← `credit_score`, `loan_amount` *(Stage 3) → generated Python code:* `interest_rate` *routed to LLM ancestral-sampling (two drivers);* `loan_amount` *FK column routed to deterministic JOIN inverse (Stage 4) → sample of materialized rows verifying tiered-lookup constraint.]*

Listing 1: Python code excerpt generated by ScribbleDB for the Credit Risk database. The `sample_interest_rate` function implements the LLM-synthesised inverse for the tiered-lookup constraint.

```
# [Placeholder: Python code excerpt]
# Expected content:
#   def sample_interest_rate(credit_score,
    loan_amount):
#       """Inverse function synthesised by LLM
    agent (Alg. 11).
```

*[Figure 8 placeholder (full-width, two panels). **Left**: schema generation latency (s) vs. number of tables. Two lines: ScribbleDB (with 1σ shading over test cases) and SchemaAgent. **Right**: per-row generation time (ms) vs. target row count n for Handcrafted dataset; separate lines for deterministic-only nodes and LLM nodes.]*

Figure 8: Scalability. Left: ScribbleDB's parallel sharding yields 3–4× lower latency than SchemaAgent on large schemas. Right: per-row generation time is constant across row counts.

[Figure 9 placeholder: Stacked bar chart. X-axis: schema-size bins (small, medium, large). Y-axis: fraction of DAG column nodes. Two stacks: deterministic-path nodes (blue) and LLM-path nodes (orange). Show that larger schemas have higher DPR (more single-dependency FK columns).]

Figure 9: Deterministic Path Ratio by schema size. The fraction of columns handled deterministically grows with schema size, amplifying the LLM call reduction on large schemas.

Table 5: Sample generated rows: Credit Risk, Loans table. `IntRate` satisfies the tiered-lookup constraint by construction.

LoanID	AppID	CreditScore	LoanAmt	IntRate
[placeholder rows with IntRate satisfying tiered-lookup]				

```
#       Implements tiered lookup: credit_score >
     750 and amount < 50000 -> rate = 0.035
#       ... additional tiers ...
#       """
#       ...
#
#   def sample_loan_amount(applicant_ids):
#       """Deterministic FK-assignment inverse (
    Alg. 10, JOIN case)."""
#       return np.random.choice(applicant_ids,
    size=N_LOANS, replace=True)
```

9 Discussion

9.1 Limitations

[Placeholder: (1) Assumes NL input is internally consistent; no contradiction detection or resolution. (2) Multivariate distribution support is partial—see §6.3; copula or conditional-chain design not yet finalised. (3) Deterministic inverse synthesis covers only single-dependency columns; many-to-many joins and compound multi-column aggregations fall back to LLM. (4) Context enrichment relies on web retrieval quality; domain jargon absent from public knowledge bases may still be under-specified.]

9.2 Failure Modes

[Placeholder: (a) Overly ambiguous NL where enrichment cannot recover missing distribution parameters (e.g., no public definition of a proprietary scoring model). (b) DAG cycles introduced by contradictory logical constraints that the patch agent cannot resolve within the retry budget. (c) Distribution parameter hallucination for rare distribution families (e.g., generalised extreme value distributions). (d) Smoke-test failures that cannot be repaired by the code-generation agent within 5 retries—currently causes a pipeline abort.]

9.3 Future Directions

[Placeholder: (1) Multivariate distributions: copula-based joint sampling to model cross-column correlations beyond univariate priors; Gaussian copulas as a first step. (2) Contradiction detection and resolution in NL input before fact extraction. (3) Extending deterministic code generation to multi-dependency chains (iterative inverse composition for chained aggregations). (4) Schema evolution: incrementally updating the generated schema and program as the NL description changes, without full pipeline re-execution. (5) Support for non-relational output (document stores, graph databases).]

10 Related Work

10.1 Database Generation

*[Para: Ab-initio generators (MUDD [**?**], PSDG [**?**], PDGF [**?**], Myriad [**?**]) operate from declarative specifications; powerful but require expert modeling effort. Regeneration-based approaches (DBSynth [**?**], RSGen [**?**], Hydra [**?**], SAM [**?**], DScaler [**?**], TouchStone [**?**]) exploit statistical metadata or workload traces from an existing deployment;*

Table 6: Positioning ScribbleDB against representative prior systems. ✓ = full support, ○ = partial, × = not supported.

System	NL Input	Schema Gen.	Dist. Constr.	Logic Constr.	Materialize
MUDD / PDGF / Myriad	×	✓	✓	×	✓
DBSynth / RSGen / SAM	×	×	✓	×	✓
SchemaAgent	✓	✓	×	×	×
ScribbleDB	✓	✓	✓	✓	✓

inapplicable in greenfield settings. Neither class accepts NL input or handles joint distributional and logical constraints.]

10.2 LLM-based Schema and SQL Generation

*[Para: NL-to-SQL benchmarks (Spider [**?**]) and the NL-to-SQL survey [**?**] demonstrate strong LLM capability on structured-output tasks. SchemaAgent / Text2Schema [**?**] is the closest prior work: it converts NL to a relational schema but stops short of data materialization, distributional constraints, and logical fidelity. ScribbleDB is the first system to cover the full synthesis pipeline.]*

10.3 Agentic Systems for Structured Output

[Para: Multi-agent orchestration, self-repair loops, and tool-augmented LLMs have been applied to code generation, document understanding, and data engineering. ScribbleDB adopts these patterns but adds deterministic guard rails and a correctness-by-construction materialization scheme, distinguishing it from generation-only agentic approaches.]

11 Conclusion

[Placeholder ∼100 words. Summarise ScribbleDB: a four-stage agentic pipeline converting an NL description into a validated relational DDL schema and an executable Python program for large-scale table materialization. Restate the key new contributions over the prior workshop version: deterministic inverse-function code generation for single-dependency DAG nodes and multivariate distribution support (in progress). Restate headline experimental results. Close with one forward-looking sentence on copula-based multivariate modelling and contradiction handling as the next milestones.]